

Lecture – 18

Harimohan Khatri

1

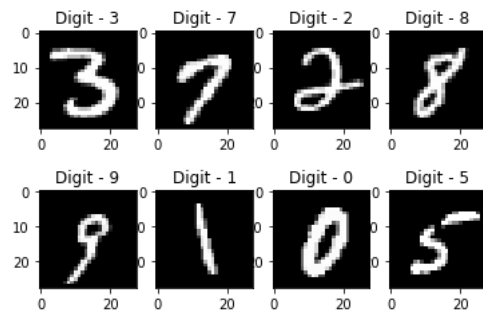
In this lecture...

- i. Image pattern and its recognition
- ii. General steps of Pattern recognition
- iii. Boundary Pre-processing, Boundary Feature Descriptors, Region Feature Descriptors
- iv. Feature extraction – PCA
- v. Scale-Invariant Feature Transform (SIFT)

2

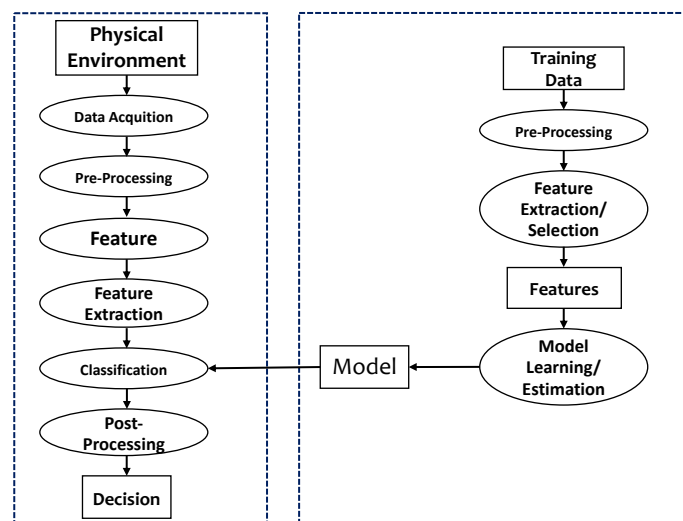
Image Patterns

- A pattern in an image refers to a repeated or structured visual feature.
- Types of Patterns:
 - Structural (based on shape/geometry)
 - Statistical (based on intensity distributions)
 - Texture-based (repeating elements or texture)
- Example: Stripes on a zebra, facial features, printed text



3

General Steps of Pattern Recognition



4

General Steps of Pattern Recognition

- Data acquisition and sensing:
 - Measurements of physical variables.
 - Important issues: bandwidth, resolution, sensitivity, distortion, SNR, latency, etc.
- Pre-processing:
 - Removal of noise in data.
 - Isolation of patterns of interest from the background.
- Feature extraction:
 - Finding a new representation in terms of features.

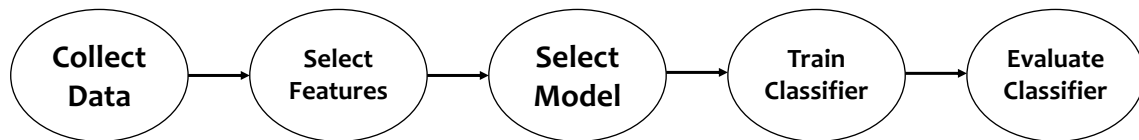
5

General Steps of Pattern Recognition

- Model learning and estimation:
 - Learning a mapping between features and pattern groups and categories.
- Classification:
 - Using features and learned models to assign a pattern to a category.
- Post-processing:
 - Evaluation of confidence in decisions.
 - Exploitation of context to improve performance.
 - Combination of experts.

6

Design Cycle of Image Recognition Systems



- **Data Collection:**

- Collecting training and testing data
- How can we know when we have adequately large and representative set of samples?

7

Design Cycle of Image Recognition Systems

- **Feature selection:**

- Domain dependence and prior information.
- Computational cost and feasibility.
- Discriminative features:
 - Similar values for similar patterns.
 - Different values for different patterns.
- Invariant features with respect to translation, rotation and scale.
- Robust features with respect to occlusion, distortion, deformation, and variations in environment.

8

Design Cycle of Image Recognition Systems

- Model selection:
 - Domain dependence and prior information.
 - Definition of design criteria.
 - Parametric vs. non-parametric models.
 - Handling of missing features.
 - Computational complexity.
 - Types of models: templates, decision-theoretic or statistical, syntactic or structural, neural, and hybrid.
 - How can we know how close we are to the true model underlying the patterns?

9

Design Cycle of Image Recognition Systems

- Training:
 - How can we learn the rule from data?
 - Supervised learning: a teacher provides a category label or cost for each pattern in the training set.
 - Unsupervised learning: the system forms clusters or natural groupings of the input patterns.
 - Reinforcement learning: no desired category is given but the teacher provides feedback to the system such as the decision is right or wrong.

10

Design Cycle of Image Recognition Systems

- Evaluation:
 - How can we estimate the performance with training samples?
 - How can we predict the performance with future data?
 - Problems of overfitting and generalization.

11

Boundary Pre-Processing

- Boundary pre-processing involves techniques like noise filtering, edge detection (e.g., using Canny or Laplacian operators), and boundary smoothing to refine object edges.
- This ensures that subsequent feature extraction is based on accurate and well-defined boundaries, reducing errors caused by noise or artifacts.
- For example, a blurred image of a face would have its edges sharpened to aid in facial recognition.

12

Feature Descriptors

- Feature descriptors in image pattern recognition are numerical representations that capture essential characteristics of an image or its regions, enabling the system to identify and classify patterns.
- They are extracted during the feature extraction stage of the life cycle and serve as inputs for classification algorithms.
- Feature descriptors summarize visual properties (e.g., shape, texture, color) into a compact form, making it easier to compare and match patterns across images despite variations in scale, rotation, or lighting.

13

Feature Descriptors

- After pre-processing (e.g., edge detection or noise removal), feature descriptors are computed.
- They are fed into the classification stage (e.g., using machine learning models) to train the system to recognize patterns.
- During evaluation, the discriminative power of these descriptors is tested to refine the system.
- They reduce dimensionality, improve robustness to variations, and enable efficient pattern matching.
- Selecting the right descriptors depends on the application (e.g., edge-based for shape recognition, texture-based for fabric analysis), and poor choices can lead to misclassification.

14

Feature Descriptors

- **Boundary Feature Descriptors:** Focus on the edges or outlines of objects.
 - Perimeter: Length of the object's boundary.
 - Curvature: Rate of change of the boundary direction.
 - Fourier Descriptors: Frequency-domain representation of the boundary shape.
- **Region Feature Descriptors:** Capture internal properties of an object's area. Examples include:
 - Area: Total number of pixels in the region.
 - Texture: Statistical measures like contrast or entropy (e.g., from Gray-Level Co-occurrence Matrix).
 - Color Histogram: Distribution of color intensities.
 - Moment Invariants: Shape descriptors invariant to translation, rotation, and scaling.

15

Feature Extraction

- Transforms raw image data into a set of meaningful features (descriptors) that capture the essential characteristics of the image or its objects.
- Reduce the dimensionality of the data while preserving relevant information for pattern recognition, making classification more efficient and effective.
- Steps:
 - Pre-processing Output: Starts with processed data
 - Feature Selection/Computation: Extract specific attributes, boundary features or region based on the application's needs.
 - Normalization: Scale or standardize features to ensure consistency (e.g., adjusting for varying image sizes).
 - Feature Vector Formation: Combine extracted features into a vector or matrix for input to classification algorithms

16

PCA – Principal Component Analysis

- Principal Component Analysis (PCA) is a linear technique for dimensionality reduction.
- Finds new axes (principal components) along directions of maximum variance in data.
- PCA on Images
 - Treat each image as a vector (e.g., $28 \times 28 = 784$ pixels).
 - Stack multiple images into a matrix X .
 - Apply PCA to:
 - Reduce each image to a lower-dimensional vector
 - Extract dominant patterns
 - Improve classification or clustering
 - PCA can reduce 784D to 50D while preserving 95% of useful information.

https://www.youtube.com/watch?v=kX_asQvluwk

17

PCA – Principal Component Analysis

Let $X \in \mathbb{R}^{n \times d}$ be data with n samples, d features.

- Center Data:

$$\bar{X} = X - \mu \quad \text{where } \mu \text{ is the mean of each column.}$$

- Covariance Matrix:

$$C = \frac{1}{n-1} \bar{X}^T \bar{X}$$

- Eigen Decomposition:

$$C v_i = \lambda_i v_i \quad \begin{array}{l} v_i: \text{eigenvectors (principal directions)} \\ \lambda_i: \text{eigenvalues (variances along directions)} \end{array}$$

- Select Top k Eigenvectors (largest λ)
- Project Data: $Z = \bar{X} \cdot V_k$ where V_k is a matrix of top k eigenvectors.

18

PCA – Principal Component Analysis

PCA for Feature Extraction

- Flatten images → vectors
- Apply PCA → transform to feature vectors
- Use feature vectors for:
 - Classification (SVM, k-NN)
 - Clustering
 - Visualization

19

Scale Invariant Feature Transform – SIFT

- SIFT is an algorithm developed by Lowe [2004] for extracting invariant features from an image.
- It is called a transform because it transforms image data into scale-invariant coordinates relative to local image features.
- SIFT features (called keypoints) are invariant to image scale and rotation, and are robust across a range of affine distortions, changes in 3-D viewpoint, noise, and changes of illumination.
- The input to SIFT is an image. Its output is an n-dimensional feature vector whose elements are the invariant feature descriptors.

20

SIFT – Scale Space

- The first stage of the SIFT algorithm is to find image locations that are invariant to scale change.
- This is achieved by searching for stable features across all possible scales, using a function of scale known as scale space, which is a multi-scale representation suitable for handling image structures at different scales in a consistent manner.
- Scale space represents an image as a one-parameter family of smoothed images, with the objective of simulating the loss of detail that would occur as the scale of an image decreases.
- The parameter controlling the smoothing is referred to as the scale parameter.
- Gaussian kernels are used to implement smoothing, so the scale parameter is the standard deviation.

21

SIFT – Scale Space

- Therefore the scale space, $L(x,y,\sigma)$ of a grayscale image, $f(x,y)$, is produced by convolving f with a variable-scale Gaussian kernel, $G(x,y)$:

$$L(x, y, \sigma) = G(x, y, \sigma) \star f(x, y)$$

- where the scale is controlled by parameter σ , and G is of the form

$$G(x, y, \sigma) = \frac{1}{2\pi\sigma^2} e^{-(x^2 + y^2)/2\sigma^2}$$

- The input image $f(x,y)$ is successively convolved with Gaussian kernels having standard deviations $\sigma, k\sigma, k^2\sigma, k^3\sigma, \dots$ to generate a “stack” of Gaussian-filtered (smoothed) images that are separated by a constant factor k .

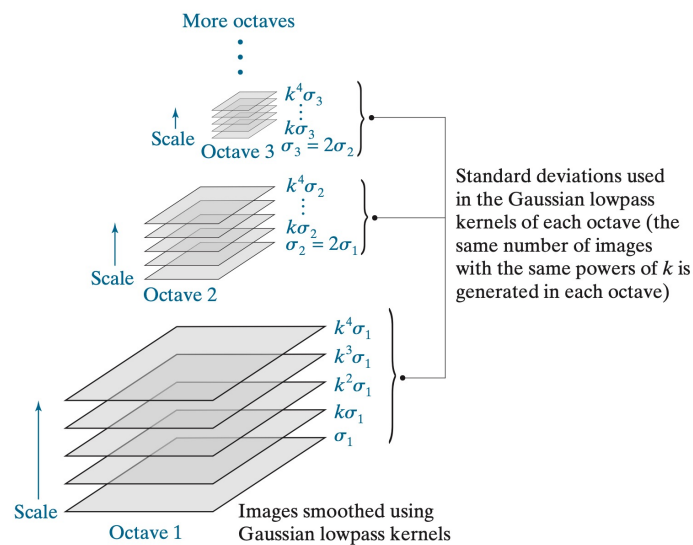
22

SIFT – Scale Space

- SIFT subdivides scale space into octaves, with each octave corresponding to a doubling of σ
- SIFT further subdivides each octave into an integer number, s , of intervals, so that an interval of 1 consists of two images, an interval of 2 consists of three images, and so forth.
- It then follows that the value used in the Gaussian kernel that generates the image corresponding to an octave is $k^s \sigma = 2 \sigma$ which means that $k = 2^{1/s}$.

23

SIFT – Scale Space



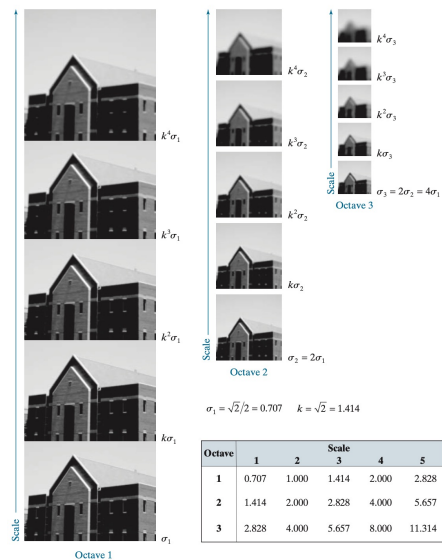
24

SIFT – Detecting Local Extrema

- SIFT initially finds the locations of keypoints using the Gaussian filtered images, then refines the locations and validity of those keypoints using two processing steps.
- **Finding the Initial Keypoints**
 - Keypoint locations in scale space are found initially by SIFT by detecting extrema in the difference of Gaussians of two adjacent scale-space images in an octave, convolved with the input image that corresponds to that octave. $D(x,y,\sigma) = [G(x,y,k\sigma) - G(x,y,\sigma)] \star f(x,y)$
$$D(x,y,\sigma) = L(x,y,k\sigma) - L(x,y,\sigma)$$
 - Difference of Gaussians is an approximation to the Laplacian of a Gaussian (LoG). $G(x,y,k\sigma) - G(x,y,\sigma) \approx (k - 1)\sigma^2\nabla^2G$

25

SIFT – Detecting Local Extrema



26

SIFT – Detecting Local Extrema

- Improving the Accuracy of Keypoint Locations
 - When a continuous function is sampled, its true maximum or minimum may actually be located between sample points.
 - This approach is used to get closer to the true extremum (to achieve subpixel accuracy) is to fit an interpolating function at each extremum point found in the digital function, then look for an improved extremum location in the interpolated function.
 - SIFT uses the linear and quadratic terms of a Taylor series expansion of $D(x,y,\sigma)$, shifted so that the origin is located at the sample point being examined. In vector form, the expression is

$$\begin{aligned} D(\mathbf{x}) &= D + \left(\frac{\partial D}{\partial \mathbf{x}}\right)^T \mathbf{x} + \frac{1}{2} \mathbf{x}^T \frac{\partial}{\partial \mathbf{x}} \left(\frac{\partial D}{\partial \mathbf{x}}\right) \mathbf{x} \\ &= D + (\nabla D)^T \mathbf{x} + \frac{1}{2} \mathbf{x}^T \mathbf{H} \mathbf{x} \end{aligned}$$

27

SIFT – Detecting Local Extrema

- where D and its derivatives are evaluated at the sample point, $\mathbf{x} = (x, y, \sigma)^T$ is the offset from that point, ∇ is the familiar gradient operator,

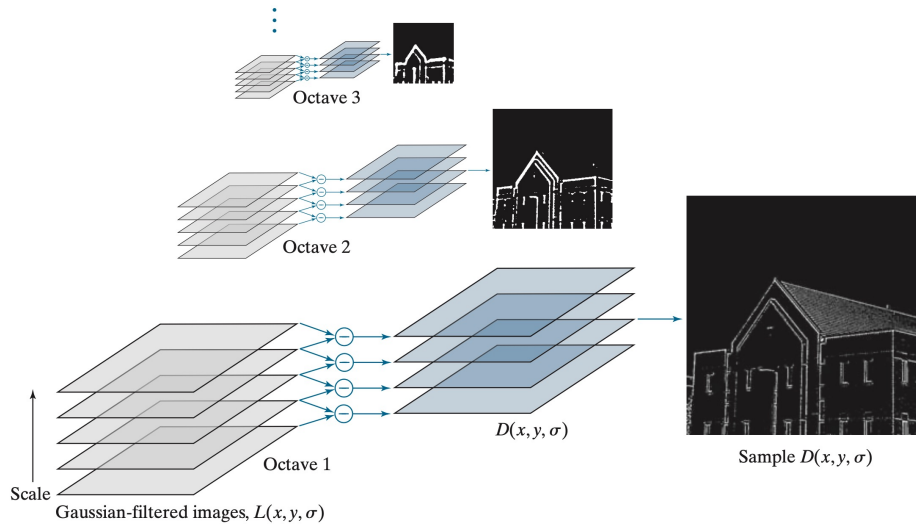
$$\nabla D = \frac{\partial D}{\partial \mathbf{x}} = \begin{bmatrix} \partial D / \partial x \\ \partial D / \partial y \\ \partial D / \partial \sigma \end{bmatrix}$$

- and $\mathbf{H}$ is the Hessian matrix

$$\mathbf{H} = \begin{bmatrix} \partial^2 D / \partial x^2 & \partial^2 D / \partial x \partial y & \partial^2 D / \partial x \partial \sigma \\ \partial^2 D / \partial y \partial x & \partial^2 D / \partial y^2 & \partial^2 D / \partial y \partial \sigma \\ \partial^2 D / \partial \sigma \partial x & \partial^2 D / \partial \sigma \partial y & \partial^2 D / \partial \sigma^2 \end{bmatrix}$$

28

SIFT – Detecting Local Extrema



29

SIFT – Detecting Local Extrema

- The location of the extremum, $\hat{x}$ is found by taking the derivative of $D(x)$, with respect to x and setting it to zero, which gives us

$$\hat{\mathbf{x}} = -\mathbf{H}^{-1}(\nabla D)$$

- If the offset $\hat{x}$ is greater than 0.5 in any of its three dimensions, we conclude that the extremum lies closer to another sample point, in which case the sample point is changed and the interpolation is performed about that point instead.
- The final offset $\hat{x}$ is added to the location of its sample point to obtain the interpolated estimate of the location of the extremum.
- The function value at the extremum, $D(\hat{x})$ is used by SIFT for rejecting unstable extrema with low contrast

$$D(\hat{\mathbf{x}}) = D + \frac{1}{2}(\nabla D)^T \hat{\mathbf{x}}$$

30

SIFT – Eliminating the Edge differences

- Keypoints of interest in SIFT are “corner-like” features. Thus, intensity transitions caused by edges are eliminated.
- To quantify the difference between edges and corners, we can look at local curvature.
- Curvature at a point in an image can be estimated from the 2×2 Hessian matrix evaluated at that point. Thus, to estimate local curvature of the DoG at any level in scalar space, we compute the Hessian matrix of D at that level:

$$\mathbf{H} = \begin{bmatrix} \partial^2 D / \partial x^2 & \partial^2 D / \partial x \partial y \\ \partial^2 D / \partial y \partial x & \partial^2 D / \partial y^2 \end{bmatrix} = \begin{bmatrix} D_{xx} & D_{xy} \\ D_{yx} & D_{yy} \end{bmatrix}$$

31

SIFT – Eliminating the Edge differences



32

SIFT – Keypoint Orientation

- Assign a consistent orientation to each keypoint based on local image properties.
- This allows us to represent a keypoint relative to its orientation and thus achieve invariance to image rotation.
 - The scale of the keypoint is used to select the Gaussian smoothed image, L , that is closest to that scale.
 - In this way, all orientation computations are performed in a scale-invariant manner. Then, for each image sample, $L(x,y)$, at this scale, we compute the gradient magnitude, $M(x,y)$ and orientation angle $\theta(x,y)$ using pixel differences:

$$M(x,y) = \left[(L(x+1,y) - L(x-1,y))^2 + (L(x,y+1) - L(x,y-1))^2 \right]^{\frac{1}{2}}$$

$$\theta(x,y) = \tan^{-1} \left[(L(x,y+1) - L(x,y-1)) / (L(x+1,y) - L(x-1,y)) \right]$$

33

SIFT – Keypoint Orientation



The keypoints superimposed on the original image. The arrows indicate keypoint orientations

34

SIFT – Keypoint Descriptor

- SIFT descriptors encode local image features around keypoints.
- Designed to be invariant to scale, rotation, illumination, and viewpoint.
- Process:
 - Extract a 16×16 region around each keypoint.
 - Compute gradient magnitude and orientation at each pixel.
 - Apply Gaussian weighting centered on the keypoint.
 - Divide region into 16 (4×4) subregions.
 - For each subregion, create an 8-bin histogram of gradient orientations.

35

SIFT – Keypoint Descriptor

Histogram & Descriptor Formation

- Each histogram bins orientations at 45° intervals.
- Gradient magnitudes are interpolated between adjacent bins to avoid abrupt boundary effects.
- $16 \text{ histograms} \times 8 \text{ bins} = 128\text{-dimensional vector}$.
- Resulting vector captures gradient distribution around keypoint.
- This vector is the SIFT descriptor, used for matching across images.

36

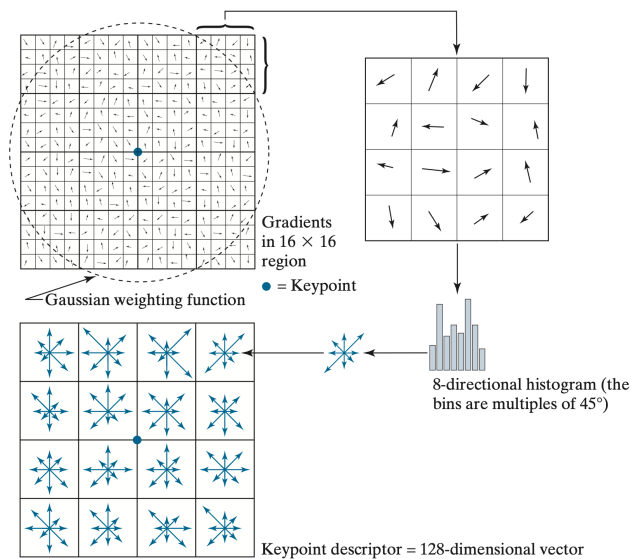
SIFT – Keypoint Descriptor

Histogram & Descriptor Formation

- Each histogram bins orientations at 45° intervals.
- Gradient magnitudes are interpolated between adjacent bins to avoid abrupt boundary effects.
- 16 histograms $\times$ 8 bins = 128-dimensional vector.
- Resulting vector captures gradient distribution around keypoint.
- This vector is the SIFT descriptor, used for matching across images.

37

SIFT – Keypoint Descriptor



38

SIFT – Keypoint Descriptor

- Rotation Invariance: Histograms are aligned relative to keypoint orientation.
- Illumination Invariance: Descriptor is L2-normalized to unit length.
- Values > 0.2 are thresholded, then vector is renormalized.
- Descriptor is robust against:
 - Linear brightness changes
 - Affine illumination variations
- Final result: a stable, distinctive descriptor suitable for reliable image matching.

39

SIFT – Summary

- **Construct the scale space.** The parameters that need to be specified are σ , s , (k is computed from s), and the number of octaves.
- **Obtain the initial keypoints.** Compute the difference of Gaussians, $D(x,y,\sigma)$ from the smoothed images in scale space.
 - Find the extrema in each $D(x,y,\sigma)$ image. These are the initial keypoints.
- **Improve the accuracy of the location of the keypoints.** Interpolate the values of $D(x,y,\sigma)$ via a Taylor expansion.

40

SIFT – Summary

- **Delete unsuitable keypoints.** Eliminate keypoints that have low contrast and/or are poorly localized. This is done by evaluating D from Step 3 at the improved Locations. All keypoints whose values of D are lower than a threshold are deleted. Keypoints associated with edges are deleted.
- **Compute keypoint orientations.** Compute the magnitude and orientation of each keypoint using the histogram-based procedure.
- **Compute keypoint descriptors.** If a region of size 16×16 around each keypoint is used, the result will be a 128-dimensional feature vector for each keypoint.

41

Discussions and Classwork

42

Lecture – 19

Harimohan Khatri

43

In this lecture...

- i. Pattern and Pattern Classes
- ii. Pattern Classification by Prototype Matching
 - i. Minimum distance classifier (nearest neighbor classifier)
 - ii. Using Correlation for prototype matching
 - iii. Matching SIFT features
 - iv. Matching structural prototypes

44

Pattern and Pattern Classes

- A pattern is a spatial arrangement of features.
- A pattern class is a set of patterns that share some common properties.
- Pattern recognition by machine encompasses techniques for automatically assigning patterns to their respective classes.
 - The job of a pattern recognition system is to assign a class label to each of its input patterns.
- There are four main stages involved in recognition:
 - Sensing,
 - Preprocessing,
 - Feature extraction, and
 - Classification.

45

Pattern and Pattern Classes

- Basic approaches used for image pattern classification:
 - Prototype matching
 - Optimum statistical classifiers, and
 - Neural networks.
- The patterns can be:
 - Labeled - we know the class of each pattern,
 - Unlabeled - data are known to be patterns, but the class of each pattern is unknown.
- There are two principal pattern arrangements:
 - Quantitative patterns: are arranged in the form of pattern vectors.
 - Structural patterns: are composed of symbols, arranged in the form of strings, trees, or graphs.

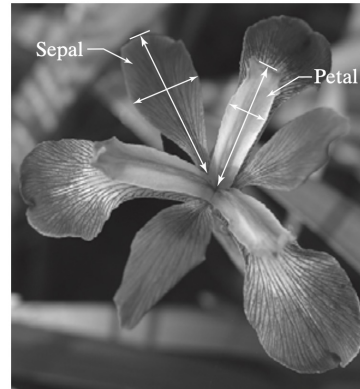
46

Pattern Vectors

Pattern vectors represents number of feature descriptors in form of,

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} \quad \begin{array}{l} x_i \text{ represents the } i^{\text{th}} \\ \text{feature descriptor, and} \\ n \text{ is the total number} \\ \text{of such descriptors.} \end{array}$$

Used in Discriminant analysis to recognize three types of iris flowers (Iris, setosa, virginica, and versicolor). Each flower was described using four features: the length and width of the petals, and similarly for the sepals.



$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix}$$

x_1 = Petal width
 x_2 = Petal length
 x_3 = Sepal width
 x_4 = Sepal length

47

Pattern Vectors Example

- Image vectors can be of signatures, boundary and region characteristics, subimages properties, movement variables, spatial bands, etc.

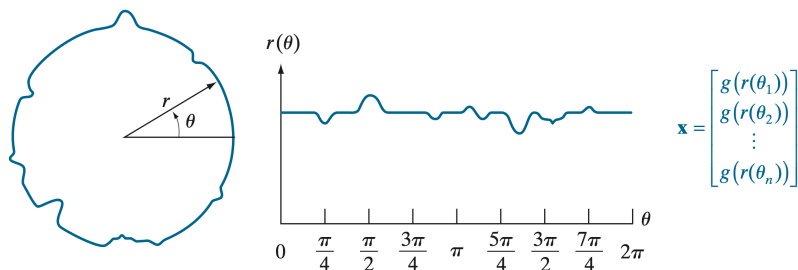


Figure: A noisy object boundary, and its corresponding signature.

48

Pattern Vectors Example

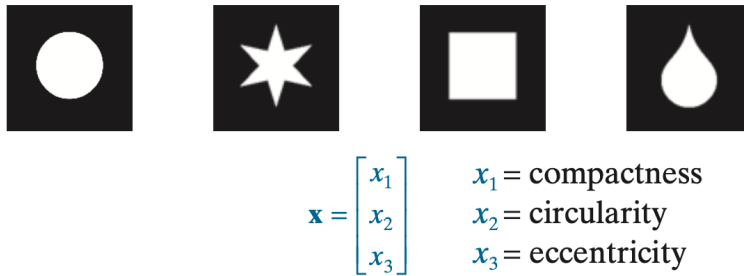


Figure: Pattern vectors whose components capture both boundary and regional characteristics

49

Pattern Vectors Example

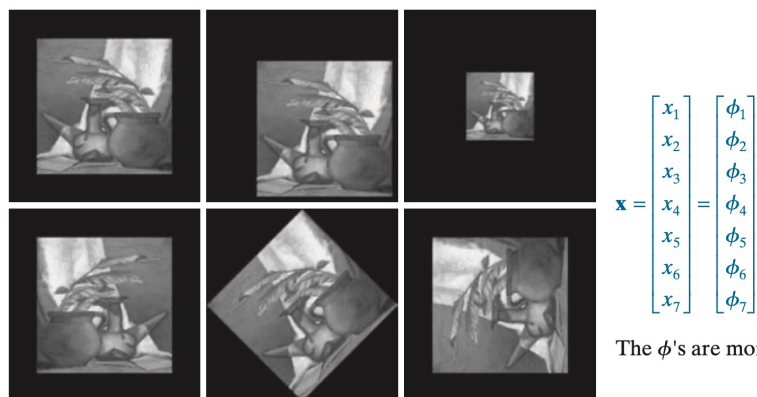
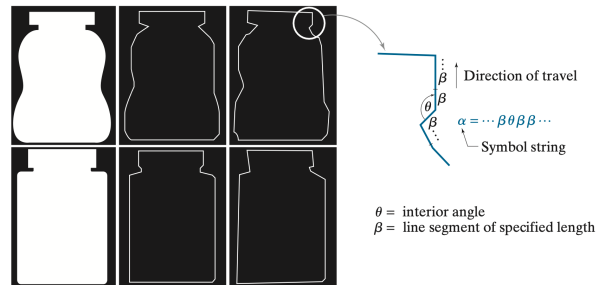


Figure: Feature vectors with components that are invariant to Transformations such as rotation, scaling, and translation. The vector components are moment invariants.

50

Structural Patterns

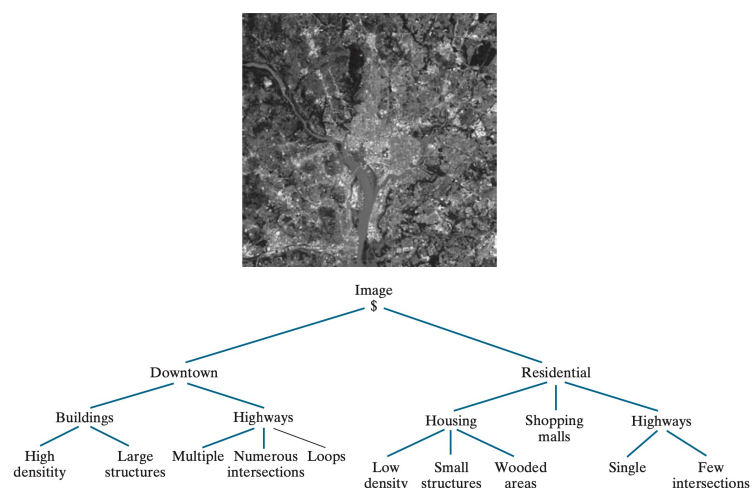
- Pattern vectors are not suitable for applications in which objects are represented by structural features, such as strings of symbols.



- Patterns containing structural descriptions of objects are important in applications where shape is of interest.

51

Tree Representation of Image Structure



52

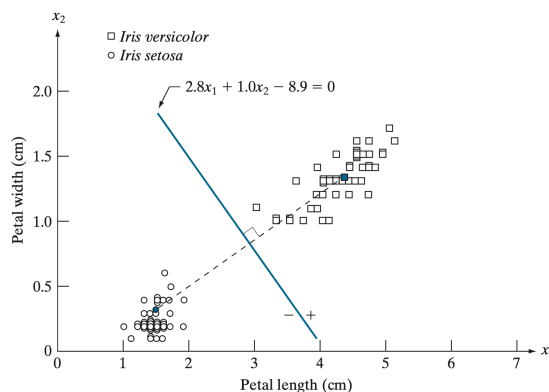
Pattern Classification by Prototype Matching

- Prototype matching involves comparing an unknown pattern against a set of prototypes, and assigning to the unknown pattern the class of the prototype that is the most “similar” to the unknown.
- Prototype matching can be implemented by using any of following:
 - Minimum distance classifier (nearest neighbor classifier)
 - Using Correlation for prototype matching
 - Matching SIFT features
 - Matching structural prototypes
 - Matching shape numbers
 - String matching

53

Minimum Distance Classifier

- It computes a distance-based measure between an unknown pattern vector and each of the class prototypes then assigns the unknown pattern to the class of its closest prototype.



→ Decision boundary of a minimum distance classifier (based on two measurements) for the classes of Iris versicolor and Iris setosa.

→ The dark dot and square are the means of the two classes.

54

- Prototype vector of the minimum-distance classifier usually are the mean vectors of the various pattern classes:

$$\mathbf{m}_j = \frac{1}{n_j} \sum_{\mathbf{x} \in c_j} \mathbf{x} \quad j = 1, 2, \dots, N_c$$

where n_j is the number of pattern vectors used to compute the j^{th} mean vector,

c_j is the j^{th} pattern class, and

N_c is the number of classes.

- If we use the Euclidean distance to determine similarity, the minimum-distance classifier computes the distances:

$$D_j(\mathbf{x}) = \|\mathbf{x} - \mathbf{m}_j\| \quad j = 1, 2, \dots, N_c$$

where $\|\mathbf{a}\| = (\mathbf{a}^T \mathbf{a})^{1/2}$ is the Euclidean norm.

$$D_j(\mathbf{x}) = \sqrt{(\mathbf{x} - \mathbf{m}_j)^T (\mathbf{x} - \mathbf{m}_j)}$$

- The classifier then assigns an unknown pattern $\mathbf{x}$ to class c_i
 - if $D_i(\mathbf{x}) < D_j(\mathbf{x})$ for $j=1, 2, \dots, N_c, j \neq i$
 - Ties $[D_i(\mathbf{x}) = D_j(\mathbf{x})]$ are resolved arbitrarily.

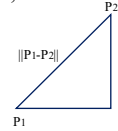
- Euclidean distance in 2D for two point $P_1(x_1, y_1)$ and $P_2(x_2, y_2)$ is given as

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

- If we define the points as vectors;

$$P_1(x_1, y_1) = \begin{bmatrix} x_1 \\ y_1 \end{bmatrix} = [x_1 \ y_1]^T$$

$$P_2(x_2, y_2) = \begin{bmatrix} x_2 \\ y_2 \end{bmatrix} = [x_2 \ y_2]^T$$



- Computing difference vector

$$P_2 - P_1 = \begin{bmatrix} x_2 - x_1 \\ y_2 - y_1 \end{bmatrix}$$

- Euclidean norm between these point vectors

$$\|\mathbf{a}\| \text{ i.e. } \|P_2 - P_1\| = \sqrt{\mathbf{a}^T \mathbf{a}} = \sqrt{(P_2 - P_1)^T (P_2 - P_1)}$$

$$d = \sqrt{[x_2 - x_1 \ y_2 - y_1] \begin{bmatrix} x_2 - x_1 \\ y_2 - y_1 \end{bmatrix}}$$

- Obtaining matrix multiplication;

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

55

- Selecting the smallest distance is equivalent to evaluating the functions

$$d_j(\mathbf{x}) = \mathbf{m}_j^T \mathbf{x} - \frac{1}{2} \mathbf{m}_j^T \mathbf{m}_j \quad j = 1, 2, \dots, N_c$$

- and assigning an unknown pattern $\mathbf{x}$ to the class whose prototype yielded the largest value of d . That is, $\mathbf{x}$ is assigned to class c_i , if

$$d_i(\mathbf{x}) > d_j(\mathbf{x}) \quad j = 1, 2, \dots, N_c; \ j \neq i$$

- Such functions are referred to as decision functions.

- The decision boundary separating class c_i from c_j is given by the values of $\mathbf{x}$ for which

$$d_i(\mathbf{x}) = d_j(\mathbf{x}) \text{ or } d_i(\mathbf{x}) - d_j(\mathbf{x}) = 0$$

- From above equation;

$$d_{ij}(\mathbf{x}) = d_i(\mathbf{x}) - d_j(\mathbf{x})$$

$$= (\mathbf{m}_i - \mathbf{m}_j)^T \mathbf{x} - \frac{1}{2} (\mathbf{m}_i - \mathbf{m}_j)^T (\mathbf{m}_i + \mathbf{m}_j) = 0$$

$$D_j(\mathbf{x}) = \sqrt{(\mathbf{x} - \mathbf{m}_j)^T (\mathbf{x} - \mathbf{m}_j)}$$

Squaring both sides,

$$D_j^2(\mathbf{x}) = (\mathbf{x} - \mathbf{m}_j)^T (\mathbf{x} - \mathbf{m}_j)$$

Performing multiplication,

$$D_j^2(\mathbf{x}) = \mathbf{x}^T \mathbf{x} - \mathbf{x}^T \mathbf{m}_j - \mathbf{m}_j^T \mathbf{x} + \mathbf{m}_j^T \mathbf{m}_j$$

$$D_j^2(\mathbf{x}) = \mathbf{x}^T \mathbf{x} - 2\mathbf{m}_j^T \mathbf{x} + \mathbf{m}_j^T \mathbf{m}_j$$

$$\text{where } \mathbf{x}^T \mathbf{m}_j = \mathbf{m}_j^T \mathbf{x}$$

Since $\mathbf{x}^T \mathbf{x}$ is same for all classes and doesnot depend on j ,

$$d_j(\mathbf{x}) = \mathbf{m}_j^T \mathbf{x} - \frac{1}{2} \mathbf{m}_j^T \mathbf{m}_j \quad j = 1, 2, \dots, N_c$$

56

Using Correlation for Prototype Matching

- The correlation process essentially involves sliding a prototype (or template) over an image and calculating a correlation coefficient at each position.
- High correlation values indicate locations where the template matches well with the image.

$$c(x, y) = \sum_{s=0}^{M-1} \sum_{t=0}^{N-1} f(x+s, y+t) \cdot w(s, t)$$

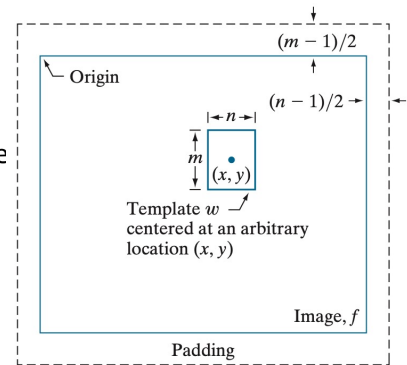
where:

c(x,y): The correlation output at position (x,y) in the image

f(x+s,y+t): The image function, evaluated at the position (x+s,y+t), which is the region of the image that is being compared with the template.

w(s,t): The prototype (template) function, which is a smaller, predefined pattern we're trying to match within the larger image.

M and N: The dimensions of the template (prototype).



57

Normalized Correlation

- Correlation can be sensitive to the brightness levels of the image and template. Normalized correlation helps mitigate this issue by accounting for brightness variations.

$$c(x, y) = \frac{\sum_{s=0}^{M-1} \sum_{t=0}^{N-1} [f(x+s, y+t) - \bar{f}_{xy}] \cdot [w(s, t) - \bar{w}]}{\sqrt{\sum_{s=0}^{M-1} \sum_{t=0}^{N-1} [f(x+s, y+t) - \bar{f}_{xy}]^2 \cdot \sum_{s=0}^{M-1} \sum_{t=0}^{N-1} [w(s, t) - \bar{w}]^2}}$$

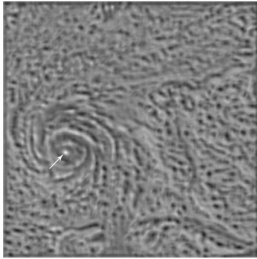
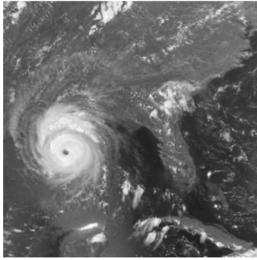
$\bar{f}_{xy}$: The mean intensity value of the region in the image corresponding to the template size M×N.

$\bar{w}$: The mean intensity value of the template.

- The numerator calculates the cross-correlation between the image region and the template, after subtracting their respective means (which centers the data and reduces the effect of brightness).
- The denominator normalizes the result, dividing by the square root of the sum of squared differences (variance) for both the image region and the template.

58

Matching by Correlation Example



- (a) 913*913 satellite image of Hurricane Andrew.
- (b) 31*31 template of the eye of the storm.
- (c) Correlation coefficient shown as an image (note the brightest point, indicated by an arrow).
- (d) Location of the best match (identified by the arrow).
This point is a single pixel, but its size was enlarged to make it easier to see.

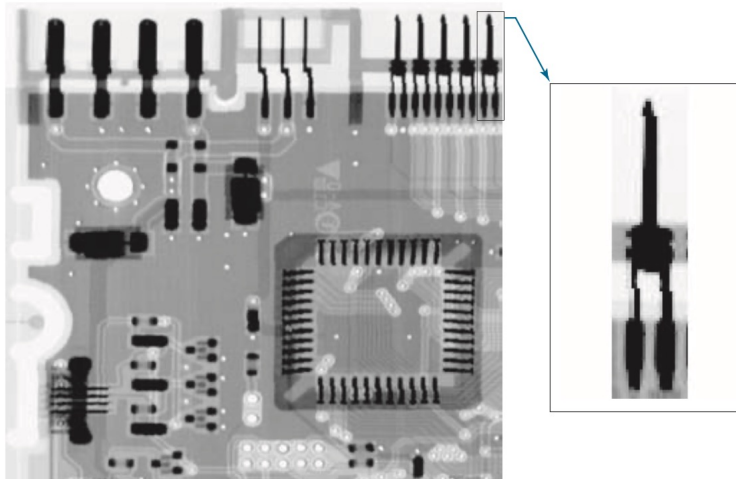
59

Matching SIFT Features

- SIFT (Scale-Invariant Feature Transform) extracts invariant features useful for matching between known (prototype) and unknown images.
- It generates 128-dimensional feature vectors for each local region in an image.
- Matching Process:
 - SIFT matches by finding correspondences between stored feature vectors of prototype and unknown images.
 - Exact matching is computationally expensive due to the large number of features.
 - To reduce computation, a best-bin-first search method is used to find likely nearest neighbors.
 - The Generalized Hough Transform helps identify clusters of potential matches, simplifying the process.

60

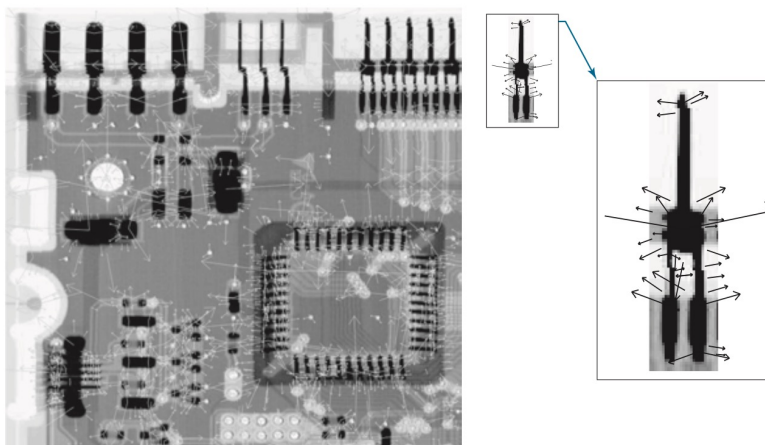
Matching SIFT Features



Circuit board image of size 948×915 pixels, and a subimage of one of the connectors. The subimage is of size 212×128 pixels, shown zoomed on the right for clarity. (Original image courtesy of Mr. Joseph E. Pascente, Lixi, Inc.)

61

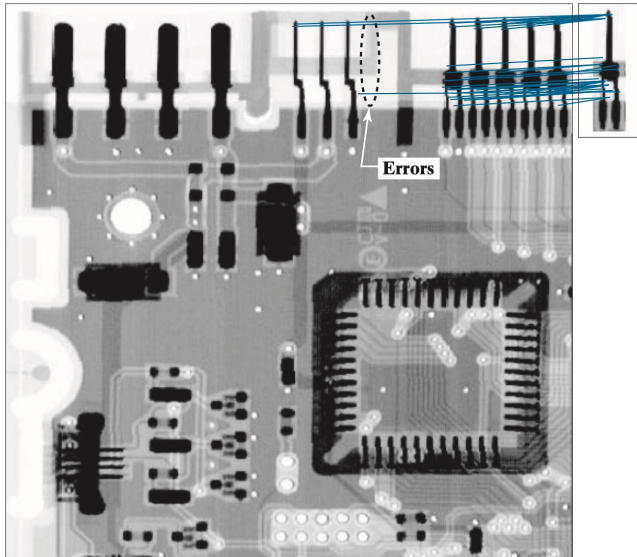
Matching SIFT Features



Keypoints found by SIFT. The large image has 2714 keypoints (visible as faint gray lines). The subimage has 35 keypoints. This is a separate image, and SIFT found its keypoints independently of the large image. The zoomed section is shown for clarity.

62

Matching SIFT Features



Matches found by SIFT between the large and small images. A total of 41 matching pairs were found. They are shown connected by straight lines. Only three of the matches were “real” errors

63

Matching Structural Prototypes

Matching Shape Numbers

- A similarity measure is defined between two shapes by comparing their shape numbers.
- For example, compare region boundaries based on shape numbers derived from 4-directional chain codes.
- Degree of Similarity (k):

- Two shapes a and b have similarity k if:

$$s_j(a) = s_j(b) \quad \text{for } j = 4, 6, 8, \dots, k; \text{ and}$$

$$s_j(a) \neq s_j(b) \quad \text{for } j = k + 2, k + 4, \dots$$

where s indicates shape number

- k is the **largest order** for which shape numbers still match.
- The moment they differ (starting from k+2), the similarity stops growing.
- So, **higher k** means **more similar** shapes.

64

Matching Structural Prototypes

Matching Shape Numbers

- Distance Function: $D(a,b) = \frac{1}{k}$
 - Properties of Distance Function:

$D(a,b) \geq 0$
 $D(a,b) = 0$ if and only if $a = b$
 $D(a,c) \leq \max[D(a,b), D(b,c)]$
- If two shapes match up to a high degree k , then their distance is **small**.
 - Identical shapes have $k=\infty$, so:
 $D(a,b) = 0$

Triangle Inequality: If a is similar to b , and b is similar to c , then a and c can't be wildly dissimilar.

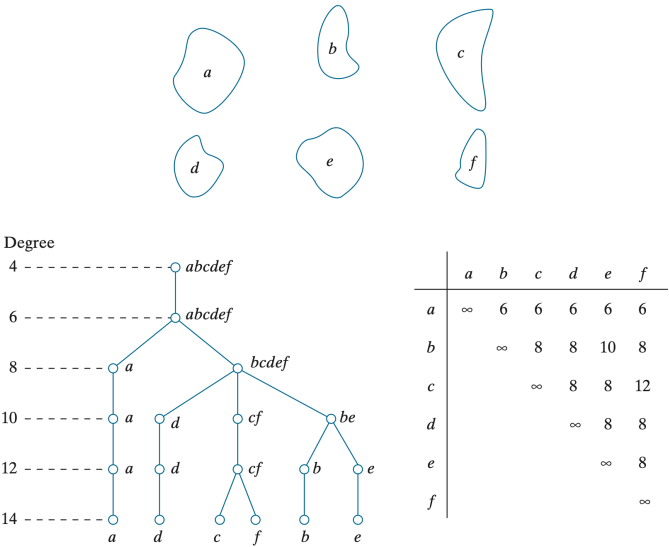
65

Matching Structural Prototypes

Visualization – Similarity Tree and Matrix

Traverse the tree by increasing degree k , grouping shapes by matching similarity.
Observations:

- Shapes are identical up to degree 8 except a , which diverges at degree 6.
- d matches up to degree 10, others continue to match.
- f and c match up to degree 12 – highest among all pairs.



66

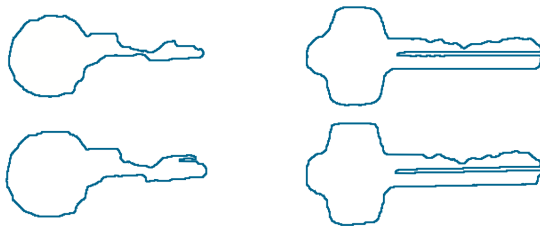
Matching Structural Prototypes

String Matching

- Let $a=a_1a_2 \dots a_n$ and $b=b_1b_2 \dots b_m$ be strings of symbols.
- α : Number of matching symbols at the same positions (i.e., $a_k=b_k$).
- β : Number of non-matching symbols $\beta = \max(|a|, |b|) - \alpha$
- Similarity Ratio (R):
$$R = \frac{\alpha}{\beta} = \frac{\alpha}{\max(|a|, |b|) - \alpha}$$
- Properties of the Similarity Ratio R:
 - $R=\infty \rightarrow$ Perfect match (identical strings)
 - $R=0 \rightarrow$ No symbols match at all
 - Higher R means greater similarity.

67

Matching Structural Prototypes



- Sample boundaries of two different object classes
- their corresponding polygonal approximations

R	1.a	1.b	1.c	1.d	1.e	1.f
1.a	∞					
1.b	16.0	∞				
1.c	9.6	26.3	∞			
1.d	5.1	8.1	10.3	∞		
1.e	4.7	7.2	10.3	14.2	∞	
1.f	4.7	7.2	10.3	8.4	23.7	∞

R	2.a	2.b	2.c	2.d	2.e	2.f
2.a	∞					
2.b	33.5	∞				
2.c	4.8	5.8	∞			
2.d	3.6	4.2	19.3	∞		
2.e	2.8	3.3	9.2	18.3	∞	
2.f	2.6	3.0	7.7	13.5	27.0	∞

R	1.a	1.b	1.c	1.d	1.e	1.f
2.a	1.24	1.50	1.32	1.47	1.55	1.48
2.b	1.18	1.43	1.32	1.47	1.55	1.48
2.c	1.02	1.18	1.19	1.32	1.39	1.48
2.d	1.02	1.18	1.19	1.32	1.29	1.40
2.e	0.93	1.07	1.08	1.19	1.24	1.25
2.f	0.89	1.02	1.02	1.24	1.22	1.18

68

Discussions and Classwork

69

Lecture – 20

Harimohan Khatri

70

In this lecture...

- i. Optimal (Bayes) Statistical Classifiers
- ii. Artificial Neural Network
 - i. Perceptron
 - ii. Hopfield Network

71

Optimum (Bayes) Statistical Classifiers

- Assign a feature vector x to one of N possible classes $\{c_1, c_2, \dots, c_N\}$
- Conditional Risk $r_k(x)$: Expected loss for assigning x to class c_j :

$$r_i(x) = \sum_{j=1}^N L_{ij} \cdot P(c_j|x)$$

L_{ij} : Loss for deciding c_i when true class is c_j

$P(c_j|x)$: Posterior probability that true class is c_j given x

- Use Bayes' theorem to compute posterior:

$$P(c_j|x) = \frac{p(x|c_j) \cdot P(c_j)}{p(x)}$$

72

Optimum (Bayes) Statistical Classifiers

- Substitute into conditional risk:

$$r_i(x) = \sum_{j=1}^N L_{ij} \cdot p(x|c_j) \cdot P(c_j)$$

- Bayes Decision Rule: Choose the class c_i that minimizes $r_i(x)$
Assign x to class c_i if $r_i(x) < r_j(x)$ for all $j \neq i$
- 0-1 Loss Matrix: Loss is 0 for correct decisions, 1 for incorrect ones.

$$L = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$$

- Simplifying Risk under 0-1 Loss $r_i(x) = \sum_{j=1}^N (1 - \delta_{ij}) \cdot P(c_j|x)$

where $\delta_{ij} = 1$ if $i=j$, and $\delta_{ij} = 0$ if $i \neq j$

73

Optimum (Bayes) Statistical Classifiers

- To minimize risk:

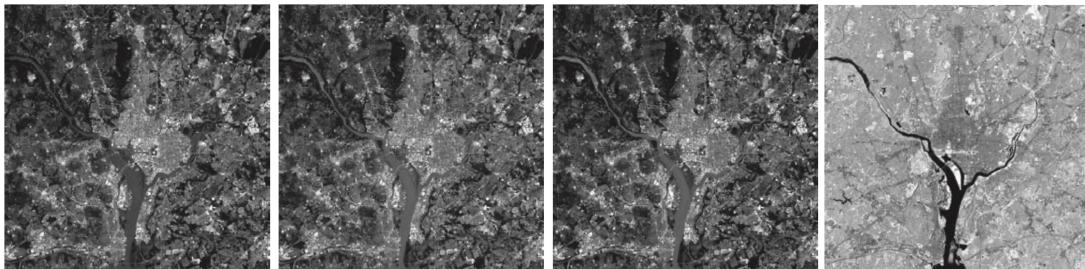
$$r_i(x) = 1 - P(c_i|x) \rightarrow \text{Maximize } P(c_i|x)$$

So, Assign x to class c_i if $P(c_i|x) > P(c_j|x)$ for all $j \neq i$

74

Bayes Classifier For Gaussian Pattern Classes

- Each class c_i is modeled by a multivariate Gaussian distribution $N(\mu_i, \Sigma_i)$, where μ_i is the mean feature vector and Σ_i is the covariance matrix capturing the spread and correlation of features within that class.



Bayes classification of multispectral data.
Images in the visible blue, visible green, visible red, and near infrared wavelength bands.

75

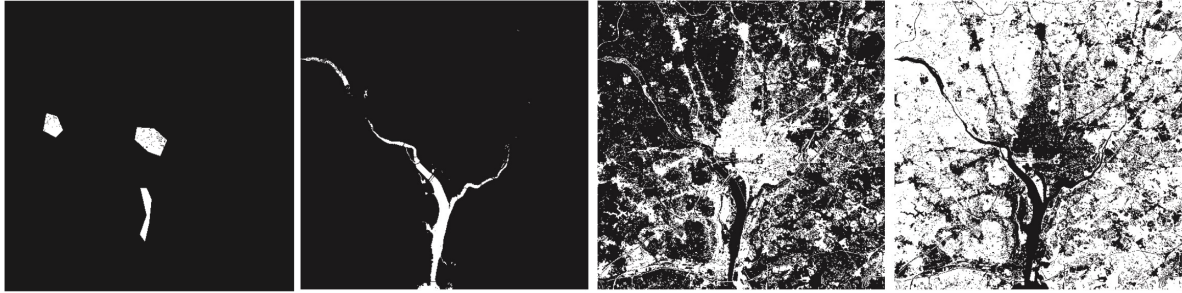
Bayes Classifier For Gaussian Pattern Classes



Masks for regions of water (labeled 1),
Urban development (labeled 2), and
Vegetation (labeled 3).

76

Bayes Classifier For Gaussian Pattern Classes



1. Results of classification; the black dots denote points classified incorrectly. The other (white) points were classified correctly.
2. All image pixels classified as water (in white).
3. All image pixels classified as urban development (in white).
4. All image pixels classified as vegetation (in white)

77

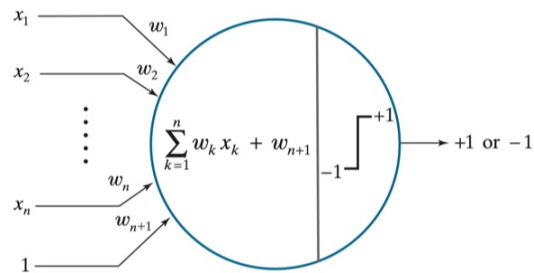
Artificial Neural Network for Pattern Recognition

- ANN is a computational model inspired by the human brain, used to recognize patterns and make predictions.
- Architecture:
 - Input Layer: Receives features (e.g., pixels, descriptors)
 - Hidden Layers: Learn patterns through weighted connections
 - Output Layer: Provides classification or decision
- Working:
 - Forward Pass: Inputs are transformed through weighted neurons.
 - Loss Computation: Error is calculated using a loss function.
 - Backpropagation: Error is propagated backward to adjust weights.
 - Optimization: Weights updated to minimize loss
- Neuron Function: $y = \sigma \left(\sum w_i x_i + b \right)$

78

Perceptron

- A perceptron takes multiple input features, applies weights, sums them, adds a bias, and passes the result through an activation function to produce a binary output.
- Perceptron Architecture
 - Inputs ($x_1, x_2, \dots, x_n$)
 - Weights ($w_1, w_2, \dots, w_n$)
 - Summation Function: computes weighted sum
 - Activation Function: usually step function
 - Output (0 or 1)



79

Perceptron

- Perceptron Learning Algorithm
 - Initialize weights and bias to small random numbers.
 - For each training example:
 - Compute prediction $\hat{y}$
 - Update weights:
 - $w_i \leftarrow w_i + \eta(y - \hat{y})x_i$
 - $b \leftarrow b + \eta(y - \hat{y})$
 - η is learning rate (e.g., 0.1)
 - Repeat until convergence

80

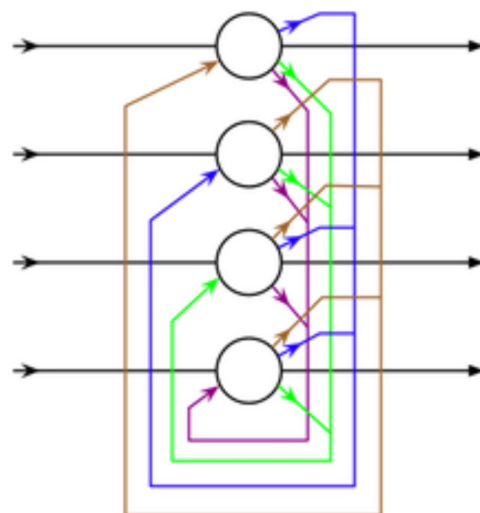
Hopfield Network

- A Hopfield network is a form of recurrent artificial neural network invented by John Hopfield.
- Hopfield nets serve as content-addressable memory systems with binary threshold units.
- They are guaranteed to converge to a local minimum, but convergence to one of the stored patterns is not guaranteed.
- Furthermore, Hopfield networks provide a model for understanding human memory.

81

Hopfield Network

- The units in Hopfield nets are binary threshold units, i.e. the units only take on two different values for their states and the value is determined by whether or not the units' input exceeds their threshold.
- Hopfield nets can either have units that take on values of 1 or -1, or units that take on values of 1 or 0.



A Hopfield net with four nodes.

82

Hopfield Network

- So, the two possible definitions for unit i's activation, a_i , are:

$$a_i \leftarrow \begin{cases} 1 & \text{if } \sum_j w_{ij}s_j > \theta_i, \\ -1 & \text{otherwise.} \end{cases}$$

$$a_i \leftarrow \begin{cases} 1 & \text{if } \sum_j w_{ij}s_j > \theta_i, \\ 0 & \text{otherwise.} \end{cases}$$

Where

$w_{ij} \rightarrow$ the strength of the connection weight from unit j to unit i

$s_j \rightarrow$ the state of unit j.

$\theta_i \rightarrow$ the threshold of unit i

83

Hopfield Network

- The connections in a Hopfield net typically have the following restrictions:
 - $w_{ii}=0, \forall i$ (no unit has a connection with itself)
 - $w_{ij}=w_{ji}, \forall i, j$ (connections are symmetric)
- The requirement that weights be symmetric is typically used, as it will guarantee that the energy function decreases monotonically while following the activation rules, and the network may exhibit some periodic or chaotic behavior if non-symmetric weights are used.
- However, Hopfield found that this chaotic behavior is confined to relatively small parts of the phase space, and does not impair the network's ability to act as a content-addressable associative memory system.

84

Hopfield Network

- Hopfield nets have a scalar value associated with each state of the network referred to as the ‘energy’, E , of the network, where:

$$E = -\frac{1}{2} \sum_{i,j} w_{ij} s_i s_j - \sum_i \theta_i s_i$$

- This value is called the ‘energy’ because the definition ensures that if units are randomly chosen to update their activations the network will converge to states which are local minima in the energy function (which is considered to be a Lyapunov function).
- Thus, if a state is a local minimum in the energy function it is a stable state for the network.

85

Discussions and Classwork

86